

AI Assisted Coding

LAB ASSIGNMENT – 3.1_1

Name: N.Trinandini

Batch: 32

Hall Ticket No: 2303A52071

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a **zero-shot prompt** (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

- Record the AI-generated code.
- Test the code with multiple inputs.

Identify any logical errors or missing edge-case handling

PROMPT:

```
#1 Write a Python code for a palindrome number check, test it with multiple inputs, and comment on any logical or edge-case issues
```

Code:

```
def is_palindrome_number(num):  
    # Convert the number to string to easily check for palindrome  
    str_num = str(num)  
    # Compare the string with its reverse  
    return str_num == str_num[::-1]  
# Test cases  
test_numbers = [121, -121, 10, 12321, 0, 1234321, 1001, 123456, "nanan", "amabba", "aba", "a"]  
for number in test_numbers:  
    result = is_palindrome_number(number)  
    print(f"Is {number} a palindrome? {result}")
```

OUTPUT:

```
Is 121 a palindrome? True  
Is -121 a palindrome? False  
Is 10 a palindrome? False  
Is 12321 a palindrome? True  
Is 0 a palindrome? True  
Is 1234321 a palindrome? True  
Is 1001 a palindrome? True  
Is 123456 a palindrome? False  
Is nanan a palindrome? True  
Is amabba a palindrome? False  
Is aba a palindrome? True  
Is a a palindrome? True
```

SUGGESTION:

Handle edge cases clearly

Ensure input validation

Test with varied inputs

Question 2: One-Shot Prompting (Factorial Calculation)

Write a **one-shot prompt** by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

PROMPT:

```
#2 Write a Python code to compute the factorial of a given number
```

CODE:

```
def factorial(n):
    if n < 0:
        return "Factorial is not defined for negative numbers"
    elif n == 0 or n == 1:
        return 1
    else:
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result
# Test cases for factorial function
test_values = [5, 0, 1, -3, 7]
for value in test_values:
    fact = factorial(value)
    print(f"The factorial of {value} is: {fact}")
```

OUTPUT:

```
The factorial of 5 is: 120
The factorial of 0 is: 1
The factorial of 1 is: 1
The factorial of -3 is: Factorial is not defined for negative numbers
The factorial of 7 is: 5040
PS C:\Users\nagas>
```

SUGGESTIONS:

Compare with zero-shot output

Check clarity of logic

Verify correctness for edge cases

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a **few-shot prompt** by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

PROMPT:

```
#3 Write a Python code to check if a number is an Armstrong number.
```

CODE:

```
def is_armstrong_number(num):
    # Convert the number to string to easily iterate over digits
    str_num = str(num)
    num_digits = len(str_num)
    sum_of_powers = sum(int(digit) ** num_digits for digit in str_num)
    return sum_of_powers == num

# Test cases for Armstrong number function
test_numbers = [153, 370, 371, 9474, 123, 0, 1, 407]
for number in test_numbers:
    result = is_armstrong_number(number)
    print(f"Is {number} an Armstrong number? {result}")
```

OUTPUT:

```
PS C:\Users\nagas> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/3_1.1.py"
Is 153 an Armstrong number? True
Is 370 an Armstrong number? True
Is 371 an Armstrong number? True
Is 9474 an Armstrong number? True
Is 123 an Armstrong number? False
Is 0 an Armstrong number? True
Is 1 an Armstrong number? True
Is 407 an Armstrong number? True
PS C:\Users\nagas>
```

SUGGESTIONS:

Observe impact of multiple examples

Evaluate code accuracy and structure

Test boundary and invalid inputs

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a **context-managed prompt** with clear instructions and constraints to generate an optimized Python program that classifies a number as **prime, composite, or neither**.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.

Compare the output with earlier prompting strategies

PROMPT:

```
#4Write a Python Code that classifies a given input number as Prime, Composite, or Neither
```

CODE:

```
def classify_number(num):
    if num <= 1:
        return "Neither Prime nor Composite"
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return "Composite"
    return "Prime"

# Test cases for classify_number function
test_numbers = [1, 2, 3, 4, 5, 16, 17, 18, 19, 20, -5, 0]
for number in test_numbers:
    classification = classify_number(number)
    print(f"The number {number} is classified as: {classification}")
```

OUTPUT:

```
PS C:\Users\nagas> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/3_1.1.py"
The number 1 is classified as: Neither Prime nor Composite
The number 2 is classified as: Prime
The number 3 is classified as: Prime
The number 4 is classified as: Composite
The number 5 is classified as: Prime
The number 16 is classified as: Composite
The number 17 is classified as: Prime
The number 18 is classified as: Composite
The number 19 is classified as: Prime
The number 20 is classified as: Composite
The number -5 is classified as: Neither Prime nor Composite
The number 0 is classified as: Neither Prime nor Composite
```

SUGGESTIONS:

- Ensure strong input validation
- Optimize logic for performance
- Compare with earlier prompting methods

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

PROMPT:

```
#5 Write a Python function to determine if a number is a perfect number
```

CODE:

```
def is_perfect_number(num):  
    if num <= 1:  
        return False  
    divisors_sum = sum(i for i in range(1, num) if num % i == 0)  
    return divisors_sum == num  
# Test cases for is_perfect_number function  
test_numbers = [6, 28, 496, 8128, 12, 97, 1, 0, -6]  
for number in test_numbers:  
    result = is_perfect_number(number)  
    print(f"Is {number} a perfect number? {result}")
```

OUTPUT:

```
Is 6 a perfect number? True  
Is 28 a perfect number? True  
Is 496 a perfect number? True  
Is 8128 a perfect number? True  
Is 12 a perfect number? False  
Is 97 a perfect number? False  
Is 1 a perfect number? False  
Is 0 a perfect number? False  
Is -6 a perfect number? False
```

SUGGESTION:

Review generated logic carefully

Test with multiple inputs

Identify missing cases or inefficiencies

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

- Input: 8 → Output: Even

- Input: 15 → Output: Odd
- Input: 0 → Output: Even

Task:

- Analyze how examples improve input handling and output clarity.
- Test the program with negative numbers and non-integer inputs.

PROMPT:

#6 Write a Python Code to find input number as "Even" or "Odd". The program must include input validation to handle non-integer values and provide clear output.

CODE:

```
def check_even_odd_static(test_values):
    for val in test_values:
        print(f"Testing input: {val}")
        try:
            # Explicitly convert to int to simulate the input behavior
            num = int(val)
            if num % 2 == 0:
                print(f"Result: The number {num} is Even.")
            else:
                print(f"Result: The number {num} is Odd.")
        except ValueError:
            print(f"Result: Invalid input '{val}'. Please enter a valid integer.")
        print("-" * 20)

# Static list of inputs to test
static_inputs = [8, 15, 0, -4, -7, "abc", 4.5]
check_even_odd_static(static_inputs)
```

OUTPUT:

```
PS C:\Users\nagas> & "C:/Program Files/Python314/python.exe" "d:/college/3-2/AIAC/AIAC LAB/3_1.1.py"
Testing input: 8
Result: The number 8 is Even.
-----
Testing input: 15
Result: The number 15 is Odd.
-----
Testing input: 0
Result: The number 0 is Even.
-----
Testing input: -4
Result: The number -4 is Even.
-----
Testing input: -7
Result: The number -7 is Odd.
-----
Testing input: abc
Result: Invalid input 'abc'. Please enter a valid integer.
-----
Testing input: 4.5
Result: The number 4 is Even.
-----
```

SUGGESTION:

Analyze effect of multiple examples

Check input validation and clarity

Test negative and non-integer cases